

LINQ Average Method in C# with Examples

In this article, I will discuss the **LINQ Average Method in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed the [LINQ Min Aggregate Method in C#](#) with Examples. The LINQ Average Method belongs to the category of Aggregate Operators. As part of this article, we will discuss the following pointers.

1. [What is the LINQ Average Method in C#?](#)
2. [Example to Understand Average Method in C#](#)
3. [LINQ Average Method with Where Extension Method using C#](#)
4. [How to use LINQ Average Method with Predicate in C#](#)
5. [LINQ Average Method with Complex Type in C#](#)
6. [When should you use the LINQ Average Method in C#?](#)

What is the LINQ Average Method in C#?

The LINQ Average method computes the average value of a numeric collection or the average of a specific numeric property in a collection of complex objects. It's part of the **System.Linq** namespace can be used with any type that implements the `IEnumerable<T>` interface, such as arrays, lists, or any collection of elements..

Example to Understand Average Method in C#:

Let us understand the LINQ Average method with Examples using C#. The following example calculates the average of all integers present in the integer collection using Query and Method syntax. The point that you need to remember is that the Average Method will work with numeric values only. In this case, the collection stores integer values so that we can apply the LINQ Average Method. We don't have any operator called average in the LINQ query syntax. So here, we need to use mixed syntax.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = new int[] { 60, 80, 50, 90, 10, 30, 70, 40, 20,
            //Using Method Syntax
            var MSAverageValue = intNumbers.Average();
            //Using Query Syntax
            var QSAverageValue = (from num in intNumbers
                                  select num).Average();
            Console.WriteLine("Average Value = " + MSAverageValue);
            Console.ReadKey();
        }
    }
}
```

Output: **Average Value = 55**

Example to Understand LINQ Average Method with Where Extension Method using C#

Let us see an example to Understand how we can use the LINQ Average Method and the Where Extension Method in C# using both Method and Query Syntax. Now, our requirement is to calculate the average of all numbers greater than 50. The following example code exactly does the same.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intNumbers = new int[] { 60, 80, 50, 90, 10, 30, 70, 40, 20,
            //Using Method Syntax
            var MSAverageValue = intNumbers.Where(num => num > 50).Average();
            //Using Query Syntax
            var QSAverageValue = (from num in intNumbers
                                   where num > 50
                                   select num).Average();
            Console.WriteLine("Average Value = " + MSAverageValue);
            Console.ReadKey();
        }
    }
}
```

Output: **Average Value = 80**

Example to Understand How to use LINQ Average Method with Predicate in C#

Let us see an example of using the LINQ Average Method with Predicate in C# using both Method and Query Syntax. Instead of using the Where Extension method to filter the data, we can also use the other overloaded version of the Average method, which takes a Predicate as a parameter. Within that predicate, we can write the logic to filter the data.

In the example below, within the Average Method, we use a Predicate and provide the condition of whether the number is greater than 50. If the number is greater than 50, then we are returning true; else, we are returning false. The following example will calculate the average of all integers greater than 50.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
```

```

static void Main(string[] args)
{
    int[] intNumbers = new int[] { 10, 30, 50, 40, 60, 20, 70, 90, 80,
    //Using Method Syntax with a Predicate
    var MSAverageValue = intNumbers.Average(num => {
        if (num > 50)
            return num;
        else
            return 0;
    });
    //Using Query Syntax with a Predicate
    var QSAverageValue = (from num in intNumbers
        select num).Average(num => {
            if (num > 50)
                return num;
            else
                return 0;
        });
    Console.WriteLine("Average Value = " + QSAverageValue);
    Console.ReadKey();
}
}
}

```

Output: **Average Value = 40**

Example to Understand LINQ Average Method with Complex Type in C#:

Let us see an example of using the LINQ Average Method with Complex Type in C# using both Method and Query Syntax. We are going to work with the following Employee class. As you can see, it is a very simple Employee class with four properties: **ID**, **Name**, **Salary**, and **Department**. Here, we also created one method, i.e., **GetAllEmployees()**, which will return the list of all the employees, which will be our data source.

```

using System.Collections.Generic;
namespace LINQDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int Salary { get; set; }
        public string Department { get; set; }
        public static List<Employee> GetAllEmployees()
        {
            List<Employee> listStudents = new List<Employee>()
            {
                new Employee{ID= 101,Name = "Preety", Salary = 10000, Department = "Sales"},
                new Employee{ID= 102,Name = "Priyanka", Salary = 15000, Department = "Sales"},
            }
        }
    }
}

```

```

        new Employee{ID= 103,Name = "James", Salary = 50000, Department =
"Sales"},
        new Employee{ID= 104,Name = "Hina", Salary = 20000, Department =
"IT"},
        new Employee{ID= 105,Name = "Anurag", Salary = 30000, Department =
"Sales"},
        new Employee{ID= 106,Name = "Sara", Salary = 25000, Department =
"Sales"},
        new Employee{ID= 107,Name = "Pranaya", Salary = 35000, Department =
"Sales"},
        new Employee{ID= 108,Name = "Manoj", Salary = 11000, Department =
"Sales"},
        new Employee{ID= 109,Name = "Sam", Salary = 45000, Department =
"Sales"},
        new Employee{ID= 110,Name = "Saurav", Salary = 25000, Department =
"Sales"}
    };
    return listStudents;
}
}
}

```

Now, our requirement is to calculate the Average Salaries of all the Employees. The following example calculates the Average Salaries of all employees using the LINQ Average method with both Method and Query Syntax. Here, we specify the numeric Salary column using a lambda expression to the Average method.

```

using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MSAverageSalary = Employee.GetAllEmployees()
                .Average(emp => emp.Salary);

            //Using Query Syntax
            var QSAverageSalary = (from emp in Employee.GetAllEmployees()
                                   select emp).Average(e => e.Salary);
            Console.WriteLine("Average Salary = " + MSAverageSalary);
            Console.ReadKey();
        }
    }
}

```

Output: **Average Salary = 26600**

Example using Average and Where Extension Method in C#:

Let us see an example of using both LINQ Average and Where Extension Methods with Complex Type using Method and Query Syntax. Our requirement is to calculate the average salary of all the employees who belong to the IT department. The following example exactly does the same. Using the Where Extension

Method, we are filtering the IT department employees, and using the Average method, we are specifying the Salary numeric column, which will calculate the average salary of IT department employees.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MSAverageSalary = Employee.GetAllEmployees()
                .Where(emp => emp.Department == "IT")
                .Average(emp => emp.Salary);

            //Using Query Syntax
            var QSAverageSalary = (from emp in Employee.GetAllEmployees()
                                   where emp.Department == "IT"
                                   select emp).Average(e => e.Salary);

            Console.WriteLine("IT Department Average Salary = " + MSAverageSalary);
            Console.ReadKey();
        }
    }
}
```

Output: IT Department Average Salary = 24000

Let's rewrite the previous example using the custom predicate. As you can see, within the Average method, we are checking the Department property value, and if the Department is It, we are returning the corresponding Salary; else, we are returning null.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax and Predicate
            var TotalSalaryMS = Employee.GetAllEmployees()
                .Average(emp => {
                    if (emp.Department == "IT")
                        return emp.Salary;
                    else
                        return null;
                });

            Console.WriteLine("IT Department Total Salary = " + TotalSalaryMS);
            Console.ReadKey();
        }
    }
}
```

```
}
```

Output: **IT Department Average Salary = 24000**

Handling Nullable Types

The Average method also has overloads for handling nullable numeric types (int?, double?, etc.). The Average method will ignore null values. For a better understanding, please have a look at the below example.

```
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int?[] nullableNumbers = { 10, 4, 2, null, 4, 5 };
            double? average = nullableNumbers.Average();
            Console.WriteLine("Average = " + average);
            Console.ReadKey();
        }
    }
}
```

Output: **Average = 5**

When should you use the LINQ Average Method in C#?

Here are some common situations when you should use the Average method:

- **Calculating Mean Value:** The primary use of Average is calculating the mean of numeric types (like int, double, decimal) in a collection. This is useful in statistics and data analysis.
- **Average of Specific Properties:** When working with a collection of custom objects, you can use Average with a lambda expression to compute the average of a specific property. For example, finding the average salary of a list of employees.
- **Handling Nullable Types:** LINQ's Average method can handle nullable numeric types (int?, double?, etc.). It will ignore null values in the computation and return null if all elements are null or if the collection is empty.
- **Database Queries:** In LINQ to Entities or LINQ to SQL, the Average method can be used to calculate the average of a column in a database table.
- **Complex Data Aggregation:** In more complex data processing scenarios, Average can be used in conjunction with other LINQ methods like GroupBy to calculate averages within groups of data.
- **Time-based Data:** While less common, Average can be used to compute the average of a set of time spans or even dates if appropriately converted to a numeric representation (like ticks).
- **Conditional Averages:** You can combine Average with Where to calculate the average of elements that meet certain conditions.

In the next article, I will discuss the [LINQ Count Method in C#](#) with Examples. I hope you understand the use and need of the **LINQ Average Method in C#** with Examples.